

软件过程改进方案设计文档

项目名称：校园活动/社群聚合与匹配平台

完成人：曾琪

日期：2026年6月

一、引言

本报告基于《AI技术应用场景匹配报告》中识别的三个关键改进环节，结合本项目的实际技术栈（Java Servlet + JSP + JDBC、MySQL、MVC架构）和开发流程（4人团队、8周迭代），按照作业要求设计AI技术与传统过程的融合方案。以下从过程重构、工具链选型、风险评估三个方面展开。

二、过程重构

（一）新增AI相关角色

在原有团队分工（项目经理、后端开发、前端开发、测试）基础上，增设两个由现有成员兼任的角色，不改变团队规模。

1. AI协调员（由项目经理兼任）

职责包括：维护项目专用的AI提示词模板库，例如针对需求访谈纪要生成用户故事的模板、针对DAO层代码生成的模板；记录每次AI调用的元数据（模型版本、提示词、输出摘要），以便问题回溯；组织每周一次的AI输出物评审会议，重点检查AI生成的需求文档和测试用例；评估AI工具的实际效率提升是否达到预期指标。

2. 验证分析师（由测试人员兼任）

职责包括：制定AI生成测试用例的验收标准（如分支覆盖率不低于85%、断言合理性）；对AI生成的JUnit测试类进行人工抽样审查；处理CI流水线中AI相关测试的失败案例，判别是代码缺陷还是AI误判；维护“AI失效案例库”，记录模型频繁出错的场景（如对日期格式的误判、对级联删除约束的遗漏），用于优化提示词或补充人工用例。

（二）调整原有活动流程

以下按三个改进环节分别说明，每个环节保留原项目实际活动，并明确AI工具执行节点和人工监督节点。

1. 需求分析阶段

原有流程为：与用户（学生、组织者）进行访谈，手写用户故事及功能点列表，团队评审后形成需求基线。改进后，访谈录音或会议纪要经脱敏处理后输入大语言模型，使用定制提示词生成用户故事初稿，格式统一为“作为...我想要...以便...”，并附带正常及异常验收标准。AI协调员将生成结果投影展示，组织团队进行逐条评审，标记不合理内容，补充非功能性需求（如并发量、响应时间）及跨故事约束。评审通过后将用户故事录入Trello看板，每个故事关联唯一ID，并与后续测试用例建立追溯关系。此环节中AI仅作为草稿生成器，不替代人工判断。

2. 编码实现阶段

原有流程为：开发人员依据设计文档手写实体类、DAO接口及其实现（如TeamRequestDaoImpl）、Servlet控制器（如TeamApplyServlet）、JSP页面，经人工走查后提交至Git。改进后，开发人员在IDE中编写方法签名及关键注释，例如“根据状态查询组队需求列表”，GitHub Copilot自动生成方法体，包括SQL语句和结果集映射。对于CRUD密集的DAO层，Copilot可根据表结构自动生成增删改查模板。开发人员本地编译并执行冒烟测试，确保生成代码无语法错误且基本逻辑正确。提交PR前触发SonarQube AI插件进行静态分析，检测潜在的SQL注入、空指针、资源未关闭等缺陷。AI协调员组织每周一次的代码审查抽查，重点关注AI生成但被开发人员直接采纳的部分，确认业务语义正确性。所有AI生成的代码在提交注释中必须包含“AI:copilot”标签。

3. 测试验证阶段

原有流程为：测试人员手写JUnit测试类（针对DAO和Service层），使用Postman手动构造接口请求并断言响应，每次迭代末期执行回归测试，人工分析失败用例。改进后，从项目已有的Swagger/OpenAPI注解导出API文档，输入Diffblue Cover工具，自动生成针对Servlet接口的JUnit测试类，覆盖正常请求、参数缺失、权限校验等场景。验证分析师审查生成的测试类，补充边界值用例（如负数ID、超长字符串），删除冗余或无效断言。将测试用例纳入GitHub Actions CI流水线，每次代码推送自动执行。当代码变更时，使用自研Python脚本调用GPT API分析变更范围，智能选择受影响的测试子集执行，而非全量运行。测试报告自动标记AI生成用例的通过率，验证分析师对失败用例进行根因分类（AI误判、代码缺陷、环境问题），定期向团队反馈AI工具的可靠性。

（三）定义AI输出物的质量标准与验证机制

针对本项目特点，制定以下可量化的质量标准。

需求阶段：每个AI生成的用户故事必须包含至少一条负面验收标准，如“搜索关键词为空时应显示默认列表”。评审会议中团队需记录AI输出中被驳回或修改的比例，目标低于20%。项目结束时通过需求追溯矩阵检查，测试用例未覆盖的需求项应反向标记为需求阶段缺陷，目标遗漏率不超过10%。

编码阶段：AI生成的代码必须通过SonarQube设定的“零严重级别警告”规则，包括安全、性能、可靠性三类。每千行代码缺陷密度目标不超过1.5，以本团队历史项目基线2.2为参照。所有包含“AI:copilot”标签的代码，在每周抽查中应有至少95%能够被开发人员正确解释其逻辑。

测试阶段：分支覆盖率不低于85%，核心模块如TeamApplyServlet和TeamRequestServiceImpl需达到90%。AI生成的测试用例首次通过率（即无需人工修改即可正确编译并断言）不低于90%。智能回归选择后，执行时间相比全量运行减少50%以上。

验证机制：采用“三层过滤”——AI自动检查（语法、覆盖率阈值）→ 验证分析师每日抽样（随机10%）→ 每周团队集体评审（重点分析AI失效案例）。所有验证结果记录在项目Wiki中，作为后续过程改进的依据。

三、工具链选型

（一）需求分析：用户故事生成

选用ChatGPT API，备选本地部署的Qwen-7B。核心功能为根据访谈纪要生成结构化用户故事及验收标准。集成方式为开发一个简单Python脚本，读取Markdown格式的会议纪要，调用API，输出为CSV表格，脚本存放于项目 /docs/ai-scripts 目录，由AI协调员执行。预期需求整理时间从2人天缩减至0.5人天，需求遗漏率降低40%。

（二）编码实现：代码生成与审查

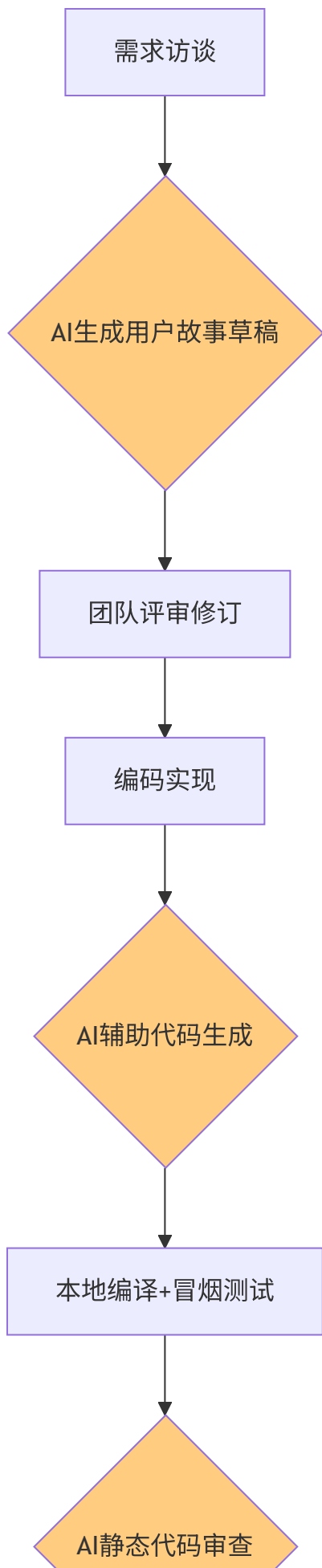
选用GitHub Copilot配合SonarQube Community Edition（带AI插件）。Copilot实时补全代码，SonarQube静态分析缺陷。Copilot作为IntelliJ IDEA插件直接使用；SonarQube通过GitHub Actions集成，每次PR自动扫描并评论。预期CRUD编码耗时减少50%，SQL注入缺陷减少60%。

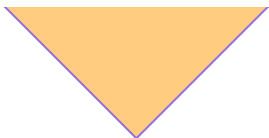
（三）测试验证：测试生成与回归优化

选用Diffblue Cover Community Edition生成JUnit测试类，辅以自研Python脚本（调用GPT API）进行智能回归测试选择。Diffblue集成到Maven的generate-sources阶段；自研脚本作为GitHub Action运行，输出需执行的测试列表。预期测试代码编写时间减少60%，分支覆盖率从65%提升至85%。

（四）改进后活动流程概览

下图展示了引入AI工具后的核心活动流程，橙色节点表示AI参与的活动：





人工抽查确认



AI生成测试用例



人工补充边界



CI自动执行测试



智能回归选择



集成部署

四、风险评估

风险一：AI生成内容存在幻觉。例如需求阶段生成不存在的功能，或代码中调用未定义的库。应对策略为严格执行人工评审关卡，所有AI输出必须经至少一名团队成员验证；建立“AI失效案例库”，每周更新并共享；提示词中加入“不要编造原始信息”等约束。

风险二：过度依赖AI导致基础技能退化。学生可能不理解AI生成的SQL或设计模式。应对策略为采用Rausch等人（2026）的建议，在迭代评审中加入口试环节，随机提问AI生成代码的原理；禁止直接粘贴未经修改的AI输出。

风险三：数据安全与隐私泄露。项目需求涉及学生个人信息。应对策略为敏感数据上传前进行脱敏处理；优先使用本地模型处理需求文档。

风险四：工具成本超支。Copilot及API调用可能产生费用。应对策略为利用GitHub Education免费获取Copilot；API调用控制每日限额，超出时使用本地模型。

五、小结

本方案针对需求分析、编码实现、测试验证三个环节，设计了AI与人工协同的改进流程，明确了新增角色（AI协调员、验证分析师）、质量标准与验证机制、工具链选型及风险评估。改进后的过程保留了人工评审的关键节点，避免对AI的盲目信任，同时通过自动化工具提升效率。下一步将在《AI技术应用验证报告》中选择测试阶段的具体场景进行原型验证，记录实际输出与效率数据。